



11/11/2024

D213 – Task 2

Performance Assessment

Lorraine Ferrusi
DR. FESTUS ELLEH

Table of Contents

PART 1: Research Question 2

 A. The Purpose of the Data Analysis..... 2

PART II: Data Preparation..... 3

 B. Data Cleaning..... 3

PART III: Network Architecture 6

 C. Describe the Type of Network..... 6

PART IV: Data Summary and Implications 8

 D. Summarize your findings and assumptions 8

PART V: Summary and Recommendations 10

 E. Neural Network Code 10

 F. Functionality and Impact 10

 G. Recommended Course of Action 11

References..... 12

PART 1: Research Question

A. The Purpose of the Data Analysis

1.

"Can movie reviews be accurately classified as positive or negative based on their text content using a neural network model?"

This involves using NLP techniques to preprocess the text data and then training a neural network model to predict the sentiment of the reviews.

2.

The following steps are the objectives of this neural network analysis (Ferrigno, 2023):

1. Data Preprocessing: Clean and preprocess the text data to make it suitable for analysis, including tokenization, removing stop words, and vectorization.
2. Exploratory Data Analysis (EDA): Perform EDA to understand the distribution of sentiments and the characteristics of the text data.
3. Model Training: Train a neural network model on the pre-processed text data to classify movie reviews as positive or negative.
4. Model Evaluation: Evaluate the performance of the trained model using metrics such as accuracy, precision, recall, and F1-score.
5. Optimization: Fine-tune the model parameters and preprocessing steps to improve classification performance.
6. Interpretation: Interpret the results to understand the key features and patterns influencing the sentiment classification.

3.

A type of neural network capable of performing a text classification task and producing useful predictions on text sequences for the selected IMDB dataset is the Recurrent Neural Network (RNN), specifically the Long Short-Term Memory (LSTM) network (Learn With Whiteboard, 2023).

LSTMs are well-suited for text classification tasks because they can capture long-term dependencies and contextual information in text sequences, making them effective for understanding and classifying sentiments in movie reviews (Coursera Staff, 2024).

PART II: Data Preparation

B. Data Cleaning

1.

The `findall` function in Python can be used to get the list of unusual characters. The `re.findall()` finds all pattern occurrences in a string and returns a list of all matches.

The `r'^\w\s,.'` expression in Python's `re` module finds all characters in the data that are not alphanumeric (`\w`), whitespace (`\s`), or a comma (`,`).

- `r'^\w\s,.'` is a raw string pattern where:
 - `^` inside the brackets means "not."
 - `\w` matches any alphanumeric character (letters and digits) and underscore.
 - `\s` matches any whitespace character (like spaces, tabs, newlines).
 - `,` matches a literal comma.

List of unusual characters: ['-', '!', '&', '"', "'", '?', ';', '/', ':', '', '(', ')', '!', '\$', '+', '%']*

The number of unique words in the data must be counted to calculate the vocabulary size. The following code processes the text from the 'Review' column in the "text_file":

- `' '.join(text_file['Review']):`
 - `text_file['Review']` extracts the 'Review' column.
 - `' '.join(...)` combines all text entries from the 'Review' column into a single, long string, with each review separated by a space.
- `.split():`
 - This splits the single combined string into individual words, creating a list of all words in the reviews.
- `set(all_words):`
 - `set(...)` removes duplicate words by creating a set from the list of all words, leaving only unique words.
- `len(set(all_words)):`
 - `len(...)` counts the number of unique words in the set, giving the vocabulary size.

*Vocabulary size: **4248***

For smaller datasets or vocabularies (fewer than 10,000 unique words), embedding sizes between 50 to 100 dimensions are often sufficient.

*Proposed embedding length: **100***

To calculate the length of each entry in the data set, the following code is used:

- `review_lengths = text_file['Review'].apply(lambda x: len(x.split()))`
 - This line calculates the number of words in each review by:
 - Splitting each review text in the Review column by spaces to get individual words.
 - Applying `len(...)` to count the words in each review.
- The result is a Series (`review_lengths`) where each entry is the word count of a review.

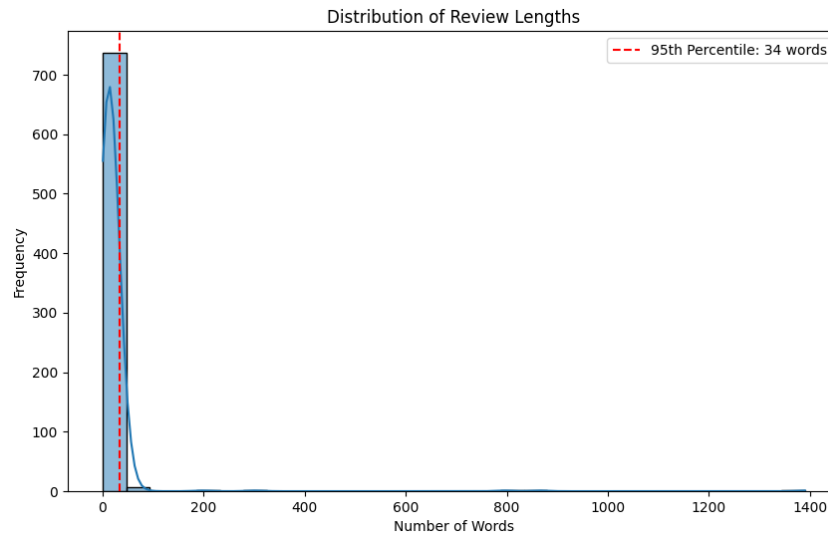
To choose the maximum sequence length, the following code is used:

- `max_sequence_length = review_lengths.quantile(0.95)`
 - This line calculates the 95th percentile of `review_lengths`, which means 95% of reviews have a length shorter than or equal to this value.

- By setting the maximum sequence length to the 95th percentile, the model can process most reviews without excessive padding while truncating only the longest ones.

The results can be visualized using a histogram and a reference line to show the 95th percentile.

Max sequence length: 34



2.

The tokenization process aims to:

- Convert Text to Tokens: Break down the text into individual units called tokens (e.g., words, subwords, or characters).
- Standardize Text: Ensure consistency by converting text to lowercase, removing punctuation, and handling special characters.
- Prepare for Model Input: Transform the text into a format that can be fed into machine learning models, such as sequences of integers or embeddings.

initialize the tokenizer

```
tokenizer = Tokenizer(num_words = 5000) # Limit to top 5000 words
tokenizer.fit_on_texts(text_file['Review'])
```

convert text to sequences of integers

```
sequences = tokenizer.texts_to_sequences(text_file['Review'])
```

pad sequences to ensure uniform length

```
max_sequence_length = 34 # As determined from EDA
```

```
padded_sequences = pad_sequences(sequences, maxlen = max_sequence_length)
```

display the first few tokenized and padded sequences

```
padded_sequences[:5]
```

```
ps_data = pd.DataFrame(padded_sequences)
```

3.

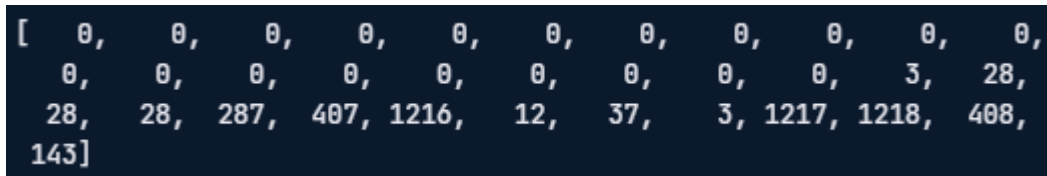
Padding is the process of adding extra tokens to sequences to ensure they all have the same length. This is crucial for feeding data into neural networks, which require inputs of uniform size.

Key Points:

- **Padding Before the Sequence:** In this case, padding tokens (usually zeros) are added to the beginning of the sequence. This is known as "pre-padding."
- **Padding After the Sequence:** Padding tokens can be added to the end of the sequence, known as "post-padding."

For this project, pre-padding was used by default, which is Keras's standard setting in the `pad_sequences` function. Below is an example screenshot of a single padded sequence. In this sequence the initial zeros represent the padding tokens, and the non-zero values represent the actual tokens from the text sequence.

This ensures that all sequences have a uniform length of 34 tokens, as determined by the EDA.



```
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 28, 28, 287, 407, 1216, 12, 37, 3, 1217, 1218, 408, 143]
```

4.

The IMDB dataset typically contains two categories of sentiment:

- Positive (usually labelled as 1)
- Negative (usually labelled as 0)

Activation Function for the Final Dense Layer:

For a binary classification problem like sentiment analysis, the sigmoid function is the appropriate activation function for the final dense layer. The sigmoid function outputs a probability value between 0 and 1, which is suitable for binary classification.

Summary:

- Number of Sentiment Categories: 2 (Positive and Negative)
- Activation Function for Final Dense Layer: Sigmoid

This setup will allow the model to output a probability indicating a positive or negative review's likelihood.

5.

Summary of Data Preparation Steps:

- **Load the Dataset:** Read and label the dataset.
- **Tokenize the Text:** Convert text reviews into sequences of integers.
- **Pad the Sequences:** Standardize the length of sequences.
- **Split the Data:** Divide the data into training, validation, and test sets.

Split the Data: Divide the data into training, validation, and test sets. The industry average split is typically (70, 15, 15)

- Training Set: 523 samples (70%)
- Validation Set: 112 samples (15%)
- Test Set: 113 samples (15%)

This ensures the model has sufficient data to learn from and separate sets to validate and test its performance.

6.

See the attached Python file: `D213 Task 2.py`

See the attached CSV file: `prepared_data.csv`

PART III: Network Architecture

C. Describe the Type of Network

1.

The model summary output:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 34, 100)	500,000
lstm (LSTM)	(None, 128)	117,248
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 1)	129

Total params: 1,852,133 (7.07 MB)
Trainable params: 617,377 (2.36 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,234,756 (4.71 MB)

2.

The model consists of 4 layers:

- Embedding Layer: Converts input sequences into fixed-sized dense vectors.
- LSTM Layer: A Long Short-Term Memory layer to capture temporal dependencies in the data.
- Dropout Layer: A regularization layer to prevent overfitting by randomly setting a fraction of input units to 0 at each update during training.
- Dense Layer: A fully connected layer with a sigmoid activation function for binary classification.

Embedding Layer: Embedding(input_dim=5000, output_dim=100, input_length=34)

1. Input Dimension: 5000 (vocabulary size)
2. Output Dimension: 100 (embedding size)
3. Input Length: 34 (maximum sequence length)

LSTM Layer: LSTM(units=128, return_sequences=False)

- Units: 128 (number of LSTM units)
- Return Sequences: False (only the output of the last LSTM unit is returned)

Dropout Layer: Dropout(rate=0.5)

- Rate: 0.5 (50% of the input units are set to 0)

Dense Layer: Dense(units=1, activation='sigmoid')

- Units: 1 (single output unit for binary classification)
- Activation: Sigmoid (for binary classification)

The total number of parameters in the model is the sum of the parameters in each layer:

- **Embedding Layer:** $(5000 \times 100 = 500,000)$ parameters
- **LSTM Layer:** $(4 \times (128 \times (100 + 128 + 1)) = 117,248)$ parameters
- **Dense Layer:** $(128 + 1 = 129)$ parameters

Total Parameters: $(500,000 + 117,248 + 129 = 617,377)$ parameters

3.

Activation Functions

- Embedding Layer: No activation function is used here as it simply converts words into fixed-sized dense vectors.
- LSTM Layer: The default activation function for LSTM is the hyperbolic tangent (tanh) for the cell state and sigmoid for the gates, which are suitable for capturing sequential dependencies.
- Dense Layer: The sigmoid activation function is used in the output layer to produce a probability score for binary classification.

Number of Nodes per Layer

- Embedding Layer: The output dimension is set to 100, which is a common choice for word embeddings, balancing between capturing semantic information and computational efficiency.
- LSTM Layer: 128 units are chosen to provide sufficient capacity to capture complex patterns in the text data without overfitting.
- Dense Layer: A single node with a sigmoid activation function is used for binary classification.

Loss Function

- Binary Cross-Entropy: This is the standard loss function for binary classification problems, measuring the difference between the predicted probabilities and the actual class labels.

Optimizer

- Adam: The Adam optimizer is chosen for its adaptive learning rate and efficient handling of sparse gradients, making it well-suited for text data and neural networks.

Stopping Criteria

- Epochs: The model is trained for 10 epochs, which is a reasonable starting point. Early stopping could be implemented to halt training when the validation loss stops improving, preventing overfitting.

Evaluation Metric

- Accuracy: Accuracy is used as the evaluation metric, which is appropriate for balanced binary classification tasks. It measures the proportion of correctly classified instances.

These hyperparameters are chosen based on common practices and empirical evidence in the field of text classification using neural networks. Adjustments can be made based on further experimentation and validation performance.

PART IV: Data Summary and Implications

D. Summarize your findings and assumptions

1.

Using stopping criteria, such as defining the number of epochs, significantly impacts the training process and the model's performance. Here are some key points to consider:

Preventing Overfitting

Early Stopping: Monitoring the validation loss and stopping training when it stops improving helps prevent overfitting. This ensures the model generalizes well to unseen data rather than memorizing the training data.

Fixed Epochs: Setting a fixed number of epochs without early stopping can lead to overfitting if the model continues to train even after the validation performance has plateaued or started to degrade.

Efficient Use of Resources

Early Stopping: Saves computational resources and time by halting training once the model has reached its optimal performance on the validation set.

Fixed Epochs: This may result in unnecessary computations if the model converges before reaching the specified number of epochs.

Model Performance

Early Stopping: Typically results in a model that performs better on the validation and test sets, as it avoids overfitting and captures the optimal training point.

Fixed Epochs: May either underfit (if the number of epochs is too low) or overfit (if the number of epochs is too high), leading to suboptimal performance.

Hyperparameter Tuning

Early Stopping: Reduces the need for extensive hyperparameter tuning related to the number of epochs, as the stopping point is dynamically determined based on validation performance.

Fixed Epochs: Requires careful tuning of the number of epochs, which can be time-consuming and may still not yield the best results.

Practical Considerations

Early Stopping: Often combined with other techniques like learning rate schedules to further enhance model performance and training efficiency.

Fixed Epochs: Simpler to implement but less adaptive to the specific characteristics of the dataset and model.

```
Epoch 1/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 52ms/step - accuracy: 1.0000 - loss: 0.0239
Epoch 2/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9766 - loss: 0.0496
Epoch 3/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9775 - loss: 0.0566
Epoch 4/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 21ms/step - accuracy: 0.9784 - loss: 0.0567
Epoch 5/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 21ms/step - accuracy: 0.9808 - loss: 0.0556
Epoch 6/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9816 - loss: 0.0549
Epoch 7/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9835 - loss: 0.0528
Epoch 8/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9840 - loss: 0.0521
Epoch 9/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 22ms/step - accuracy: 0.9845 - loss: 0.0514
Epoch 10/17 [0m 0s 32m] --- [0m 0s 37m] --- [0m 0s 1m 0s] 27ms/step - accuracy: 0.9849 - loss: 0.0508 - val_accuracy: 0.7589 - val_loss: 0.8280
Training stopped at epoch: 9
```

2.

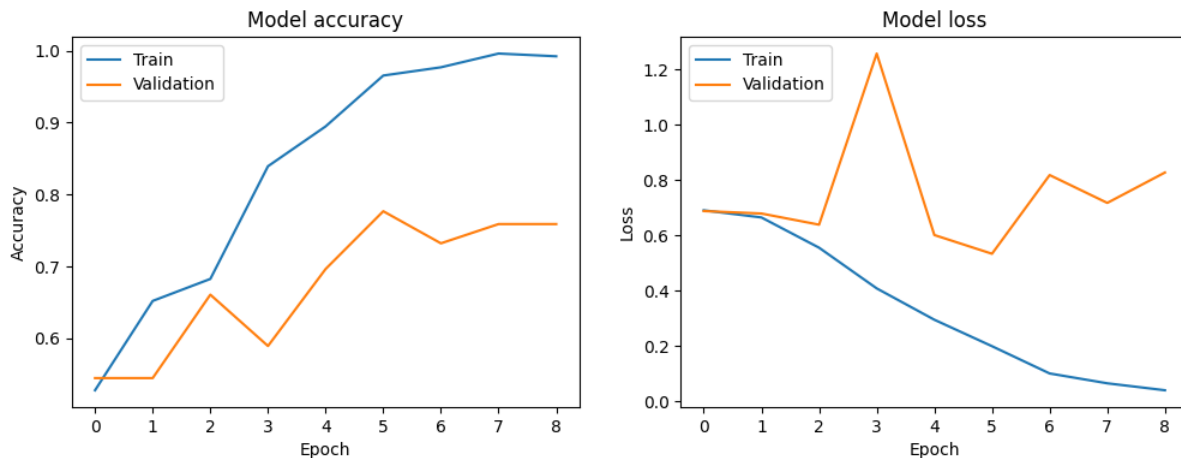
Model Fitness

- **Training and Validation Accuracy:** Examining the accuracy of the training and validation sets can determine how well the model performs.
- **Training and Validation Loss:** Monitoring the loss values can help explain if the model is learning effectively and if there is any overfitting.

Actions Taken to Address Overfitting

- **Early Stopping:** Implemented to halt training when the validation loss stops improving, preventing the model from overfitting to the training data.
- **Dropout Layer:** Added to the model to randomly drop units during training, which helps regularise the model and reduce overfitting.
- **Limited Epochs:** Training for a reasonable number of epochs (with early stopping) to avoid excessive training that could lead to overfitting.
- **Validation Split:** Using a validation set to monitor the model's performance and decide stopping criteria.

3.



- **Model Accuracy Plot (Left)**

Training Accuracy: The training accuracy increases steadily with each epoch, approaching a high value near 1.0. This suggests that the model is learning to fit the training data well.

Validation Accuracy: The validation accuracy initially improves but fluctuates after epoch 3, levelling off with only slight increases. This indicates that, beyond a certain point, the model may not be learning patterns that generalize well to the validation set.

Possible Overfitting: The growing gap between training and validation accuracy suggests overfitting, as the model continues to improve on the training data but struggles to improve on the validation data.

- **Model Loss Plot (Right)**

Training Loss: The training loss decreases consistently, which is typical as the model optimizes and learns from the training data.

Validation Loss: The validation loss initially decreases slightly but then spikes at epoch 3, after which it fluctuates without showing a clear downward trend. The spike and fluctuations in validation loss indicate that the model's generalisation ability is not improving consistently.

Overfitting Evidence: The divergence between training and validation loss after epoch 3 further suggests that the model is overfitting. While the training loss keeps decreasing, the validation loss oscillates, indicating the model is fitting noise or irrelevant patterns in the training data rather than learning generalizable features.

4.

A test accuracy of 75.22% for an RNN (Recurrent Neural Network) indicates that the model correctly predicts the target class 75.22% of the time on unseen test data.

PART V: Summary and Recommendations

E. Neural Network Code

See the attached Python file: D213 Task 2.py

F. Functionality and Impact

Functionality of the Neural Network

The neural network is designed to classify text reviews by binary sentiment. It takes sequences of words as input and outputs a binary sentiment label (positive or negative). Here's a breakdown of the functionality (Ramos, 2021):

- **Embedding Layer:** Converts words into dense vectors of fixed size. This layer helps in capturing the semantic meaning of words.
- **LSTM Layer:** Processes the sequences of word embeddings and captures temporal dependencies. LSTM (Long Short-Term Memory) networks are particularly good at handling sequential data and capturing long-range dependencies.
- **Dropout Layer:** Regularizes the model by randomly setting a fraction of input units to zero during training. This helps in preventing overfitting.
- **Dense Layer:** Outputs the final prediction. The sigmoid activation function is used to produce a probability score between 0 and 1, which is suitable for binary classification.

Impact of the Network Architecture

The architecture of the neural network has a significant impact on its performance and generalization capabilities. Here's how each component contributes (Hardesty, 2017):

Embedding Layer:

Input Dimension: The size of the vocabulary (5000 in this case). Limiting the vocabulary size helps in reducing the complexity and computational requirements.

Output Dimension: The size of the word vectors (100 in this case). A larger dimension can capture more semantic information but also increases the computational cost.

LSTM Layer:

Units: The number of LSTM units (128 in this case). More units can capture more complex patterns but also increase the risk of overfitting and computational cost.

Return Sequences: Set to False to return only the output of the last time step. This is suitable for many-to-one tasks like sentiment classification.

Dropout Layer:

Rate: The dropout rate (0.5 in this case). A higher dropout rate can help in reducing overfitting but may also slow down the learning process.

Dense Layer:

Units: The number of units (1 in this case) for binary classification.

Activation Function: Sigmoid activation function to output a probability score.

Training and Evaluation

- **Early Stopping:** Monitors the validation loss and stops training when it stops improving. This helps in preventing overfitting and ensures that the model generalizes well to unseen data.
- **Optimizer:** Adam optimizer is used for training. It is an adaptive learning rate optimization algorithm that is well-suited for this type of task.
- **Loss Function:** Binary cross-entropy loss is used, which is appropriate for binary classification tasks.

The architecture is designed to balance complexity and generalization, making it suitable for sentiment classification. Early stopping, dropout, and a well-chosen optimizer further enhance the model's performance and robustness.

G. Recommended Course of Action

75.22% accuracy demonstrates that the model has learned patterns that allow it to predict outcomes better than chance. If there's an established baseline for similar tasks (e.g., previous models or domain benchmarks), 75.22% could be considered good or poor relative to that. For instance, text sentiment analysis models often achieve 80–90% accuracy, while some time series prediction tasks may be challenging to push beyond 70%.

Generalization Ability

Decent Generalization: Test accuracy reflects the model's ability to generalize to new data, so 75.22% indicates moderate generalization. The model has likely learned some underlying trends in the data rather than just memorizing training samples, but there might still be room for improvement.

Evaluating Task-Specific Performance

Adequate for Certain Applications: For tasks where high accuracy isn't critical, 75.22% may be acceptable. For example, if the model provides recommendations or classifications in non-critical contexts, this level of accuracy might suffice.

Potentially Insufficient for High-Stakes Tasks: For applications where accuracy directly impacts outcomes (e.g., medical diagnosis, fraud detection), 75.22% might be too low, as the model could misclassify nearly 1 in 4 instances.

Interpreting Model Limitations and Next Steps

Potential Underfitting: If training accuracy was significantly higher, 75.22% could suggest that the model didn't fully capture complex patterns in the data or generalized poorly. Experimenting with a deeper or more sophisticated model architecture (e.g., a bidirectional LSTM or a transformer-based model) could improve accuracy.

Data Quality and Quantity: Accuracy might be constrained by the quality and quantity of the training data. Increasing data volume or cleaning could boost model performance if the dataset is small or noisy.

Hyperparameter Tuning: Further tuning of hyperparameters (e.g., learning rate, dropout rate, sequence length) might help the model achieve higher test accuracy.

Use Additional Metrics for a Full Evaluation

Precision, Recall, F1 Score: Accuracy alone doesn't capture the complete picture, especially if there's class imbalance. For example, if one class dominates, the model could reach high accuracy by predicting that class more often. Checking metrics like precision, recall, and F1 score can reveal if the model performs well across all classes.

Confusion Matrix: Reviewing the confusion matrix can show which classes are most often misclassified, helping you understand specific areas where the model could improve.

A test accuracy of 75.22% for an RNN is a respectable starting point demonstrating some predictive power, though it may or may not be sufficient depending on the application. To improve it, consider refining the model architecture, tuning hyperparameters, and exploring additional data sources. Complementing accuracy with other metrics will also provide a complete understanding of the model's strengths and weaknesses.

References

Coursera Staff. (2024, April). *8 Common Types of Neural Networks*. Retrieved from Coursera:
<https://www.coursera.org/in/articles/types-of-neural-networks>

Ferrigno, B. (2023, December). *Main Goals of Artificial Neural Networks*. Retrieved from Concept Marketing Group:
<https://www.marketingsource.com/post/main-goals-of-artificial-neural-networks-2>

Hardesty, L. (2017, April). *Explained: Neural networks*. Retrieved from MIT News: <https://news.mit.edu/2017/explained-neural-networks-deep-learning-0414>

Learn With Whiteboard. (2023, May). *8 Tips on How to Choose Neural Network Architecture*. Retrieved from Medium:
https://medium.com/@learnwithwhiteboard_digest/8-tips-on-how-to-choose-neural-network-architecture-e50590e99ab1#:~:text=Conclusion,of%20your%20machine%20learning%20system.

Ramos, D. (2021, July). *Real-Life and Business Applications of Neural Networks*. Retrieved from Smartsheet:
https://www.smartsheet.com/neural-network-applications?srsId=AfmBOooTrgCNO8ZktlbOMXkrC3gPgtp5YZAyKFS_xjtvR-0ueTvPglAM